

Flask: Templates & Static Files

A step-by-step guide for class

What you will learn

- Why we put HTML in **template files** instead of returning strings from Python.
- How to use `render_template()` to show an HTML page.
- How **template inheritance** works (`base.html + {% extends %} + {% block %}`).
- How to send data from Python to HTML (`{{ variable }}` and `{% for %}` loops).
- How to load CSS from the **static** folder with `url_for()`.
- Common mistakes and a short security note.

Requirements

- Python 3 and a virtual environment
- Flask installed: `pip install flask`

1. Why templates?

Until now we returned HTML directly from Python, like this:

```
@app.route("/about")
def about():
    return "<h1>About Page</h1>"
```

This works for one line, but a real page has a head, a menu, a footer and a lot of HTML. Writing all of that inside Python strings gets messy fast, and every page would repeat the same menu and footer.

Solution: keep HTML in separate **.html** files (templates) and CSS, images, and JavaScript in a **static** folder. Python only decides *which* page to show and *what data* to put in it.

2. Project structure

Create this structure. The folder names **templates** and **static** are special: Flask looks for them automatically, so spell them exactly like this.

```
myapp/
|-- app.py           <- the Flask application (Python)
|-- templates/       <- HTML files (Jinja2 templates)
|   |-- base.html    <- shared layout: head, menu, footer
|   |-- index.html   <- home page
|   `-- tools.html   <- page that shows a list
`-- static/          <- files sent to the browser as they are
    |-- css/
    `-- style.css    <- the stylesheet
```

Commands to create it from a terminal:

```
mkdir -p myapp/templates myapp/static/css
cd myapp
touch app.py templates/base.html templates/index.html \
    templates/tools.html static/css/style.css
```

3. Step 1: The Python file (app.py)

myapp/app.py

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def index():
    return render_template("index.html", title="Home")

@app.route("/tools")
def tools():
    tool_list = ["nmap", "Wireshark", "Burp Suite", "Metasploit"]
    return render_template("tools.html", title="Tools", tools=tool_list)

if __name__ == "__main__":
    app.run(debug=True)
```

What is happening?

- `render_template("index.html")` finds the file inside the **templates/** folder, fills it in, and returns the HTML.
- Extra arguments like `title="Home"` or `tools=tool_list` are **variables** the template can use.
- The name on the left (`tools=`) is the name used in the HTML. The value on the right is the Python data.

4. Step 2: The base layout (base.html)

This is the "skeleton" every page shares. We write the menu and footer **once**.

myapp/templates/base.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}My App{% endblock %}</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
  <header>
    <h1>My App</h1>
    <nav>
      <a href="{{ url_for('index') }}">Home</a>
      <a href="{{ url_for('tools') }}">Tools</a>
    </nav>
  </header>

  <main>
    {% block content %}{% endblock %}
  </main>

  <footer>
    <p>&copy; 2026 My App</p>
  </footer>
</body>
</html>
```

Key ideas

- `{% block content %}{% endblock %}` is an **empty slot**. Each child page fills it with its own content.
- `{% block title %}My App{% endblock %}` is a slot with a **default value**. If a child page does not fill it, "My App" is used.
- `url_for('static', filename='css/style.css')` builds the correct URL to the CSS file: `/static/css/style.css`.
- `url_for('index')` builds the URL to the Python **function** named `index`. If you change the route later, the links still work.

5. Step 3: A child page (index.html)

myapp/templates/index.html

```
{% extends "base.html" %}

{% block title %}{{ title }}{% endblock %}

{% block content %}
    <h2>Welcome</h2>
    <p>This page is rendered from a Jinja2 template that extends base.html.</p>
{% endblock %}
```

- `{% extends "base.html" %}` means "start with base.html". It must be the **first line**.
- We only write the blocks we want to fill. Everything else comes from base.html.
- `{{ title }}` prints the value we passed from Python (`title="Home"`).

6. Step 4: Sending a list and using a loop (tools.html)

myapp/templates/tools.html

```
{% extends "base.html" %}

{% block title %}{{ title }}{% endblock %}

{% block content %}
    <h2>Security Tools</h2>
    <ul>
        {% for tool in tools %}
            <li>{{ tool }}</li>
        {% endfor %}
    </ul>
{% endblock %}
```

Jinja2 syntax cheat sheet:

Syntax	Meaning	Example
<code>{{ ... }}</code>	Print a value	<code>{{ title }}</code>
<code>{% ... %}</code>	Logic (for, if, block, extends)	<code>{% for t in tools %}</code>
<code>{# ... #}</code>	Comment (not sent to the browser)	<code>{# TODO #}</code>

The loop repeats the `<li>` line once per item. The browser receives plain HTML:

What the browser receives (View Page Source)

```
<ul>
  <li>nmap</li>
  <li>Wireshark</li>
  <li>Burp Suite</li>
  <li>Metasploit</li>
</ul>
```

7. Step 5: The static folder (style.css)

Files in **static/** are not processed by Jinja2. Flask sends them to the browser as they are. Put CSS, images, and JavaScript here.

myapp/static/css/style.css

```
body {
    font-family: Arial, sans-serif;
    margin: 0;
    background-color: #f4f4f4;
    color: #333;
}

header, footer {
    background-color: #222;
    color: #fff;
    padding: 1rem;
    text-align: center;
}

main {
    max-width: 800px;
    margin: 2rem auto;
    padding: 0 1rem;
}

nav a {
    color: #4fc3f7;
    margin: 0 0.5rem;
    text-decoration: none;
}
```

How the CSS reaches the browser:

```
static/css/style.css    (file on disk)
    |
url_for('static', filename='css/style.css')
    |
/static/css/style.css   (URL the browser requests)
```

Try it: open `http://127.0.0.1:5000/static/css/style.css` in the browser. You will see the CSS file itself. That proves Flask is serving the static folder.

Adding an image works the same way. Save it as `static/img/logo.png` and use:

```

```

8. Step 6: Run the app

```
source venv/bin/activate      # activate your virtual environment
pip install flask             # only the first time
cd myapp
python3 app.py
```

Open these URLs in the browser:

- <http://127.0.0.1:5000/> - home page
- <http://127.0.0.1:5000/tools> - list rendered with a for loop

Tip: with `debug=True` the server restarts when you save `app.py`. If a CSS change does not appear, refresh with **Ctrl+Shift+R**. The browser caches static files.

9. Common mistakes

Problem	Cause / Fix
<code>TemplateNotFound:</code> <code>index.html</code>	The folder is not named exactly templates , or it is not next to <code>app.py</code> . You can also get this error if you run python from a different folder.
Page shows, but no styles	Wrong path in <code>filename=</code> , the folder is not named static , or the browser cache. Check the browser DevTools (F12) Network tab for a 404 error.
The literal text <code>{{ title }}</code> appears	You opened the <code>.html</code> file directly in the browser. Templates only work when Flask serves them through a URL.
<code>BuildError</code> in <code>url_for</code>	<code>url_for</code> uses the function name (<code>'tools'</code>), not the URL (<code>'/tools'</code>).
Child page shows nothing	The block name in the child does not match <code>base.html</code> (<code>content</code> vs <code>contents</code>).

10. Security note: auto-escaping

Jinja2 **escapes** variables in `.html` templates automatically. If a user sends `<script>alert(1)</script>`, the browser shows it as text and does not run it. This protects the page against **XSS** (Cross-Site Scripting).

```
{{ user_input }}          <- safe: HTML is escaped
{{ user_input | safe }}    <- DANGEROUS: turns escaping off

render_template_string(user_input) <- DANGEROUS: template injection (SSTI)
```

Rule: never use `| safe` or `render_template_string()` with data that comes from a user.

11. Practice exercises

Exercise 1: About page

Add a route `/about` that renders a new template `about.html`. It should extend `base.html`. Add a link to it in the menu of `base.html`.

Exercise 2: Pass more data

In `/about`, send your name and course from Python: `render_template("about.html", name="Ana", course="Ethical Hacking")`. Show both on the page.

Exercise 3: A conditional

In `tools.html`, show the message "No tools yet" if the list is empty. Use `{% if tools %} ... {% else %} ... {% endif %}`.

Exercise 4: Style it

In `style.css`, give the `li` items a border and some padding. Change the header color.

Exercise 5: Add an image

Put an image in `static/img/` and show it on the home page using `url_for`.

Exercise 6 (challenge): A list of dictionaries

Send a list of tools where each one is a dictionary, and display it as an HTML table:

```
tools = [
    {"name": "nmap", "use": "Network scanning"},
    {"name": "Wireshark", "use": "Packet analysis"},
]

{# in the template: #}
{% for t in tools %}
    <tr><td>{{ t.name }}</td><td>{{ t.use }}</td></tr>
{% endfor %}
```